

Some Network Acronyms

LAN Local Area Network. Your House. Private.

Wireless LAN LAN over WiFi.

WAN Wide Area Network. Assembly of LANs, e.g. a university

The Internet Public assembly of WANs

VLAN Virtual Local Area Network. Logical partitioning of LAN

VPN Virtual Private Network. Private network over a public net.

Virtual network requires cryptography

Telco TELEphone COmpany

wifi: generated from Wireless Ethernet Compatibility Alliance, WECA

Cable Layer

A **cross section** of the shore-end of a modern submarine communications cable.

- 1  – Polyethylene
- 2  – Mylar tape
- 3  – Stranded steel wires
- 4  – Aluminium water barrier
- 5  – Polycarbonate
- 6  – Copper or aluminium tube
- 7  – Petroleum jelly
- 8  – Optical fibers

Wan:

WAN Latencies

WAN Ping Demo

WAN Routers

Internet routers transmit data across the physical network connections from source to destination

Internet Protocol

The Internet Protocol suite specifies:

Host addressing.

Data transmission formats.

Message routing and delivery characteristics.

Bonus:

These lost to TCP/IP and are not commercially relevant

Token-rings

IBM's SNA

TCP/IP Stack

OSI Model	TCP/IP Protocol Suite					
Application	File Transfer	Web Browser	Email	Remote Login	Name Resolution	IP Address
Presentation						
Session						
	FTP TFTP	HTTP	SMTP IMAP POP3	Telnet	DNS	DHCP
Transport	Transmission Control Protocol TCP			User Datagram Protocol UDP		
Network	Internet Protocol IP			ARP ICMP		
Data Link	Ethernet	Token Ring		FDDI	WAN Protocols	
Physical	Copper Twisted Pair					
	Fiber Optic			Wireless		

IP Suite Layers

- Data link layer: specifies communications across a single network segment. Implemented by device drivers and network cards inside machines.
- Internet (IP) layer: specifies addressing and routing protocols for traffic to traverse the independently managed networks that comprise the Internet (and private networks).
- Transport (TCP, UDP) layer: specifies protocols for reliable (TCP) and best-effort (UDP) host-to-host communications.

- Application layer: specifies application-level protocols such as HTTP and FTP.

5G

5G uses **higher frequency radio waves** (especially mmWave for the fastest speeds):

- Higher frequencies = more bandwidth = faster speeds
- But higher frequencies don't penetrate buildings well
- And they attenuate (weaken) much faster over distance

High in frequency, but need more infrastructure to support.

Network Layer: Internet Protocol (IP)

ID for users.(informally)

- Well, every Network Interface Card (NIC) has a unique IP address
- Most machines only have one NIC
- Exception: High-end servers and Network Attached Storage often have multiple NICs

Domain Naming System (DNS)

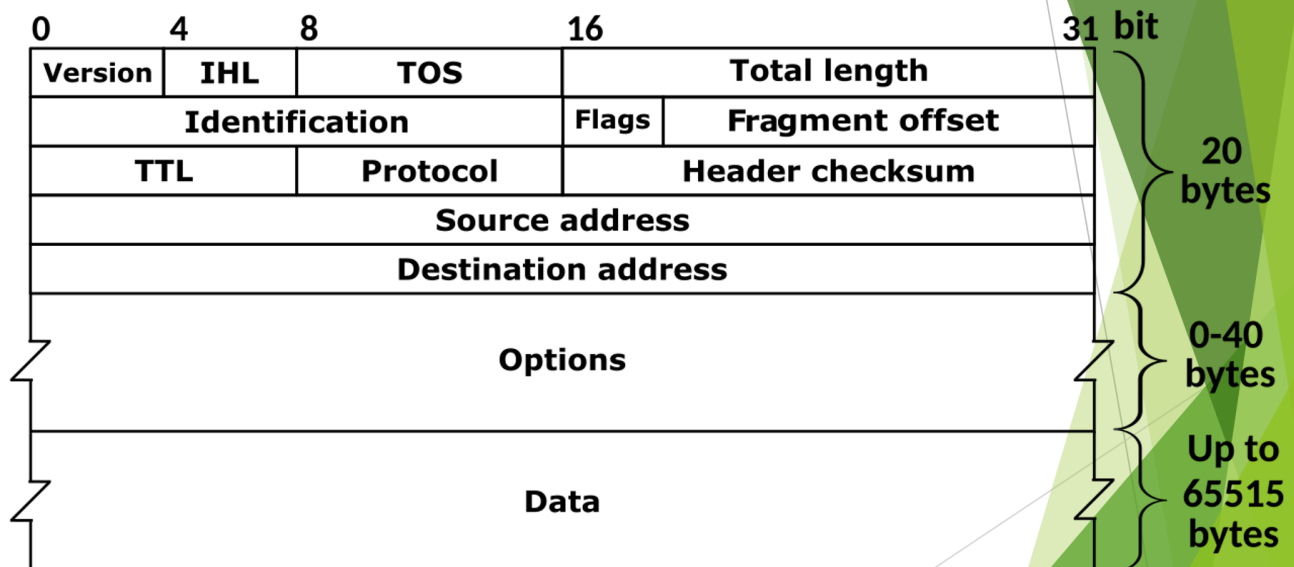
A **global hash of names** to IP addresses

Every host on the public Internet has a unique IP address (*)

Use nslookup

northeastern.edu is 155.33.17.68

IP Packet Structure



Problems:

Doesn't recover from transmission errors such as:

- Data corruption
- Packet loss
- Duplication

Also, even for packets they have some problems:

- Results in out-of-order delivery to the receiver
- Allows network to dynamically respond to conditions such as link failure and congestion
- Wifi/Cell handover

So IP is unreliable

UDP is fast but unreliable(User Datagram Protocol):

Modify:

- No acknowledgements from server
- Thin layer on top of the underlying IP protocol
- Trades off raw performance over reliability

for Streaming audio, movies, video conferencing, and gaming; it's ok
for File downloads, remote procedure calls, most applications we need

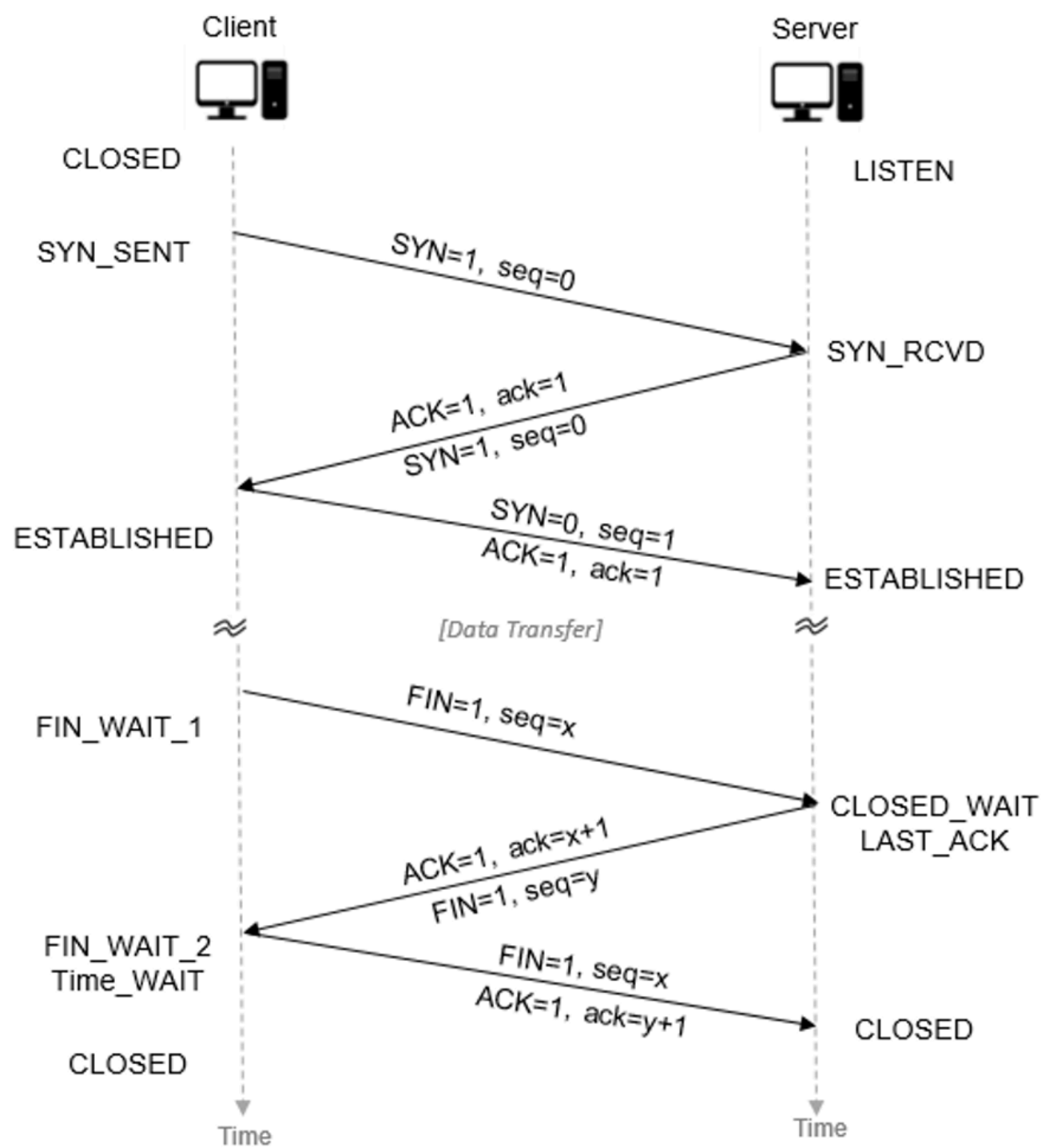
TCP is slower but reliable (more useful)

- Connection-oriented (like a phone call)
- Stream-oriented (like a phone call)
- Reliable (either your message gets through, or you get an error)

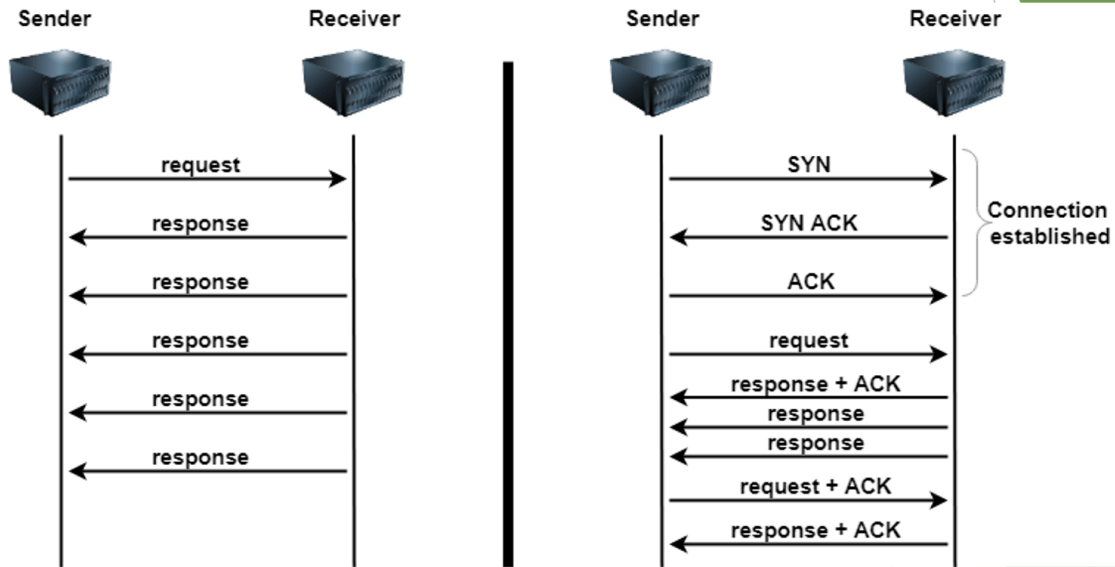
3-step handshake to establish connection between client and server

1. sender: you want?
2. Receiver: I want!
3. Sender: Receive I want and start to send

TODO:



UDP vs TCP traffic comparison



Feature

- checksums to check packet integrity
- dynamic flow control

TCP Socket

A **socket** is an **endpoint for sending and receiving data** across a network.

- A **specific connection endpoint** that uses TCP
- A programming interface that allows applications to use TCP
- Combines: IP address + port number + connection state
- It's the actual implementation/object you work with in code

TCP VS TCP socket

- **TCP** = The telephone protocol/system (how calls work, dial tones, ringing, voice transmission rules)
- **TCP socket** = Your actual phone handset that you use to make calls

Tips:

- **.com** = commercial
- **.org** = organization (non-profits)
- **.net** = network (internet service providers, network infrastructure)

- **.edu** = education (educational institutions)
- **.gov** = government (US government)
- **.mil** = military (US military)

CAP theorem

In [database theory](#), the **CAP theorem**, also named **Brewer's theorem** after computer scientist [Eric Brewer](#), states that any [distributed data store](#) can provide at most [two of the following three](#) guarantees:¹(https://en.wikipedia.org/wiki/CAP_theorem#cite_note-Gilbert_Lynch-1)[²](https://en.wikipedia.org/wiki/CAP_theorem#cite_note-2)[³](https://en.wikipedia.org/wiki/CAP_theorem#cite_note-3)

[Consistency](#)

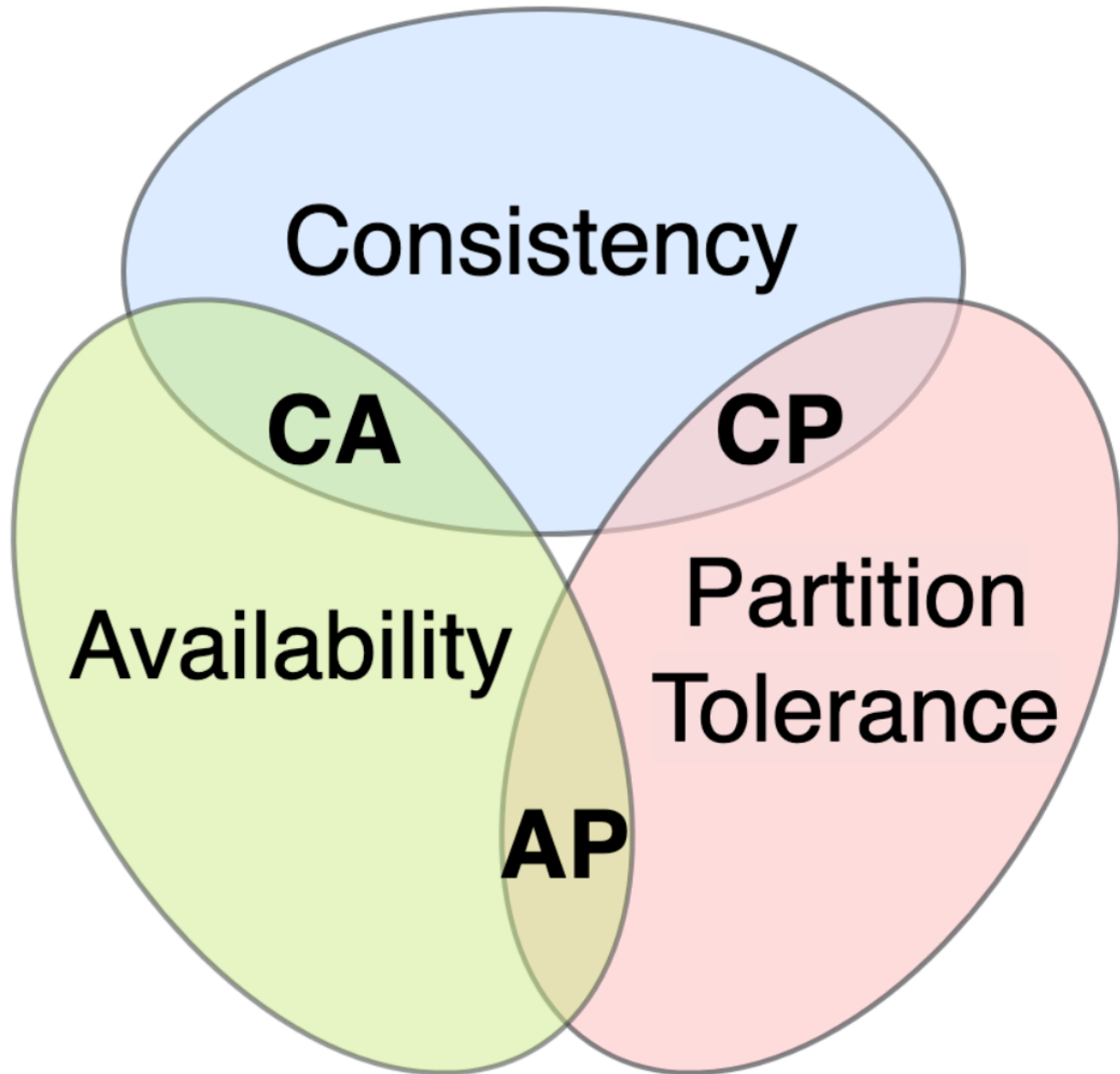
Every read receives the most recent write or an error. Consistency means that all clients see the same data at the same time, no matter which node they connect to. For this to happen, whenever data is written to one node, it must be instantly forwarded or [replicated](#) to all the other nodes in the system before the write is deemed 'successful'⁴(https://en.wikipedia.org/wiki/CAP_theorem#cite_note-:1-4). Consistency as defined in the CAP theorem is quite different from the consistency guaranteed in [ACID database transactions](#).⁵(https://en.wikipedia.org/wiki/CAP_theorem#cite_note-5)

[Availability](#)

Every request received by a non-failing node in the system must result in a response, without the guarantee that it contains the most recent version of the data⁶(https://en.wikipedia.org/wiki/CAP_theorem#cite_note-:3-6). This is the definition of availability in CAP theorem as defined by Gilbert and Lynch.¹(https://en.wikipedia.org/wiki/CAP_theorem#cite_note-Gilbert_Lynch-1) Availability as defined in CAP theorem is different from [high availability](#) in software architecture.⁷(https://en.wikipedia.org/wiki/CAP_theorem#cite_note-Fowler_2015-7)

[Partition tolerance](#)

The system continues to operate despite an arbitrary number of messages being dropped (or delayed) by the network between nodes.



IP Addresses

Two versions

IPv4

32-bit addresses, written as “four octets” 97.113.122.65

No longer enough addresses for the whole world

Still the most common

IPv6

128-bit addresses

Will “always” have sufficient numbers for the entire galaxy ☐

Slow adoption

Ignored in this course

No two machines (actually, network cards) can have the same address
Internet Assigned Numbers Authority (IANA) registers the IP addresses

The subranges only to be used within private networks

192.168.0.0/16 (CIDR format)

192.168.0.0 through 192.168.255.255

Also written 192.168.x.x

10.0.0.0/8

10.0.0.0 through 10.255.255.255

Also written 10.x.x.x

CIDR Format – the number after the slash is the number of fixed bits. Every 8 bits is one of the four parts of the full IP number

IP address and Port

“Well known” services have defined port numbers

22 for ssh

80 for http

443 for https

8080 is Tomcat (& Spring Boot) default for http, so that it doesn't accidentally block any local web server on port 80

- **IP address** = The building's street address (gets you to the building)
- **Port** = The apartment number (gets you to the specific apartment)

Firewall: not all the info. can pass

Sits between the OS and the network

Only allows traffic in and out on defined ports

Conceptually very simple

AWS Security Groups are firewalls

Network Address Translation (NATing)

A NAT server allows private addresses (10.x.x.x) inside a network by translating between public addresses to the outside world, and private addresses on the inside

It names the info. with the useable IP addresses and a randomly assigned port to let the outsider recognizes you.

VPN

Virtual Private Networks (VPN)

A way of sending encrypted traffic from the client to the server, then the server sends it on using NATing

Cryptography for Secure Networks

ISP: Internet Server Provider

For the security of the network, we need to crypt. the info. we want to send out to the internet.

All current cryptographic technologies rely on secret keys

A key is a long sequence of bits

Symmetric keys – Both nodes have the same key

Fast to encode and decode

Problem – how does Alice safely send the key to her friend Bob?

Asymmetric keys – Each node has one key in a pair

Public Key Encryption PKI, Diffie-Helman algorithm

SSH, HTTPS, Signal use PKI

Slow to execute. PKI usually used to exchange a symmetric key.

https (port 443) uses PKI, http (port 80) does not

Remark: Asymmetric keys need public, private keys to decode the info. ,
but symmetric keys only need public key.

Remote Procedure call:

Synchronous (i.e. Blocking, the client waits for the answer or it times out)

The client calls it

The client waits

The function call returns a value

Synchronous VS Parallel

- **Synchronous + Sequential:** Traditional single-threaded program (one thing at a time, in order)
- **Synchronous + Parallel:** Multiple threads that coordinate with each other
- **Asynchronous + Sequential:** One task at a time, but can switch between tasks while waiting
- **Asynchronous + Parallel:** Multiple independent tasks running simultaneously

Remote Method Invocation (Java RMI) (1)

Distributed Communications

“Invoking” (calling) a method on a remote machine

TCP socket create a connection between client and server which exchange data over that connection

Client might request a balance for their checking account

{“balance”, “000169990”}

Server responds with account number and current balance

{“000169990”, “220.77”}

Application Protocol

: Doing “login”, “transfer”, “address”, “statement”, “transactions”, etc.

application specific protocol: Doing specific assignments.

Application Protocol Interface (API): implementation

RMI/RPC

(RMI is a **specific implementation** of RPC for Java)

Advantages:

- Location transparency
- Clear contract specification using an API
- Type-checking

Disadvantages:

- Marshalling and Unmarshalling can become inefficient
Marshalling: the process of converting data into a format that can be sent over a network.
- Tight coupling of client to server
Change in API requires recompiling every client
- Cross language marshalling and unmarshalling is complex
Replaced by HTTP (port 80)

Important HTTP Response Codes

200 OK – everything is Okay

201 Created – OK, and specifically the POST or PUT succeeded

400 Bad Request – Bad data from the client

403 Forbidden – Security error, client probably needs to log in

404 Not Found – URL does not resolve to anything

500 Internal Server Error – The default error, uncaught exception

502 Bad Gateway – A server behind this one failed. Load balancers often return 502

503 Service Unavailable – The service is dead Jim

For the replacement:

HTTP Microservice Protocols replaced RPC mechanisms such as Java RMI

Web Servers

1. History: Web Servers and Application servers

Static-only web servers

cgibin and Perl (historical interest only)

Java servlets

Common web server components

Common web servers by language

Static

```
public class StaticServer {  
    public static void main(String[] args) {  
        String contentRoot = args[1]; ServerSocket serverSocket = new  
ServerSocket(80);  
        BufferedReader in = serverSocket.getInputStream();  
        String filename; while ((filename = in.readLine()) != null) {  
            // Open file contentRoot + filename  
            // Read file contents and write to the socket  
        }  
    }  
}
```

cgibin, connect to the SQL now:

```
  
try (ServerSocket serverSocket = new ServerSocket(80)) {  
    BufferedReader in = // blah  
    String filename; while ((filename =  
in.readLine()) != null) {  
        // Open file contentRoot + filename  
        // Read file contents and write to the socket  
        if filename.startsWith("cgibin") {  
            program_name = // rest of string following cgibin  
            response_string = System.run(f"{cgi_bin_dir}/{program_name}")  
            // write response_string to Socket  
        }  
    }  
    The program in cgi_bin contains SQL to query the database and returns the  
    result to stdout  
}
```

Contemporary Web Servers: new techniques

Tomcat is from late 90's

SpringBoot has an embedded Tomcat web server

A Route Mapping System

Configured by Xml, json, or usually function annotations

A Security System

URLs are within authentication domains

Threads or Processes

Tomcat uses a ThreadPool within one process

Flask uses threads via WSGI

Web.xml vs Annotations

Web.xml maps a URL to the code that generates the html

Infrastructure as Code (IaC) Terraform

Manually clicking to create VMs and Networks is:

- Slow
- Boring
- Error Prone
- Not Repeatable, especially during a Sev-1 at 3am

```
provider "aws" {
  region = "us-west-2"
}

data "aws_ami" "ubuntu" {
  most_recent = true

  filter {
    name = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }

  owners = ["099720109477"] # Canonical
}

resource "aws_instance" "app_server" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"

  tags = {
    Name = "learn-terraform"
  }
}
```

```
}  
}
```

This is **Terraform** configuration code — a popular Infrastructure as Code (IaC) tool.

What it does:

- Defines an AWS EC2 instance (virtual server) as code
- Specifies: region (us-west-2), Ubuntu AML image, t2.micro instance type
- When you run `terraform apply`, it automatically creates this server in AWS

TODO: still don't know what is AWS EC2

1. Slow → Fast

Before IaC: Manually click through AWS console, configure settings, wait, repeat for each server
With IaC: Write code once, run `terraform apply`, creates hundreds of servers in minutes

2. Boring → Automated

Before IaC: Repetitive clicking, filling forms, checking boxes over and over

With IaC: Write the config once, computer does the boring work forever

3. Error Prone → Consistent

Scenario: Creating a web server that should NOT be exposed to the internet

Before IaC (Manual clicking in AWS Console):

Engineer A creates a server at 2pm:

1. Clicks "Launch Instance"
2. Selects Ubuntu
3. Selects t2.micro
4. **Forgets to configure security group properly**
5. Leaves default: Port 22 (SSH) open to 0.0.0.0/0 (entire internet!)
6. Server is now hackable from anywhere 🚨

Engineer B creates another server at 5pm:

1. Same steps...
2. Remembers security group

3. But types `192.168.1.0/23` instead of `192.168.1.0/24` (typo!)
4. Wrong subnet - some legitimate users can't access it

Result: Two servers, configured differently, both wrong in different ways.

```
resource "aws_instance" "web_server" {
  ami          = "ami-12345"
  instance_type = "t2.micro"

  vpc_security_group_ids = [aws_security_group.web_sg.id]

  tags = {
    Name = "web-server"
  }
}

resource "aws_security_group" "web_sg" {
  name = "web-server-sg"

  # Only allow traffic from our office
  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["192.168.1.0/24"] # Our office network only
  }
}
```

What happens:

1. **Code review:** Team reviews this code before it runs
 - "Hey, is `192.168.1.0/24` correct?"
 - "Yes, that's our office network"
 - Security group is correct 
2. **Engineer A runs it:** Creates server with correct security group
3. **Engineer B runs it:** Creates **identical** server with same security group
4. **Engineer C at 3am:** Runs same code, gets **exactly the same result**

Result: Every server is configured identically and correctly

4. Not Repeatable → Perfectly Repeatable

Short Summary: IaC is like a **blueprint** that remembers everything

The 3am Sev-1 scenario:

Before IaC:

- 3am: Production server crashed
- Groggy engineer logs into AWS console
- "Wait, what settings did we use? What size? Which subnet? Security groups?"
- Frantically searches documentation or tries to remember
- Takes 30+ minutes, might get it wrong

With IaC:

bash

```
# 3am: Server crashed
terraform apply # Rebuilds identical server in 2 minutes
# Back to sleep
```

The code is **version controlled** (Git), **documented**, and **executable** — it IS the documentation.

Choosable IaC tools:

Terraform

OpenTofu open source is a fork from 1.5.3

Puppet

Chef

Ansible

Kubernetes

Doesn't define the system below the k8s server, but does define everything above

Terraform VS K8s

Terraform = Infrastructure Provisioning

Creates the infrastructure itself (servers, networks, databases, cloud resources)

Kubernetes (K8s) = Container Orchestration

Manages applications running on that infrastructure (deploys, scales, monitors containers)

HTTP VS HTML

HTML stands for **HyperText Markup Language** — it's the standard language for creating web pages.

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Page</title>
</head>
<body>
  <h1>Welcome to My Website</h1>
  <p>This is a paragraph of text.</p>
  <a href="https://example.com">Click here</a>
  
</body>
</html>
```